

School of Computer Science, McGill University

COMP-512 Distributed Systems, Fall 2026

Programming Assignment 1: Distributing an Application*

Due date: Report is due October 5, Demo dates the following 3 days

Your task in this programming assignment is to develop a component-based distributed information system experimenting with different communication paradigms.

You can achieve a total of 100 Points on this assignment. That is the number of points reflects the weight of each of the deliverables within this assignment.

1 Implementation (75%)

This programming assignment is a Travel Reservation system, where customers can reserve flights, cars and rooms for their vacation. You can find more information about the client interface and explore the functionality in the `clientUserGuide.pdf`.

The functionality of the management system is very simple, but it will make the programming easier. We make the following simplifications:

1. There are only 3 types of reservable items (key denoted in *italics*):
 - (a) **Flights:** *flight number*, price, and the number of seats
 - (b) **Cars:** *location*, price, and the amount available
 - (c) **Rooms:** *location*, price, and the amount available

Note that since location is used as a key, there is only one type of car/room (and only one price) for a particular location.

2. Adding an duplicate item updates the price (if greater than zero) for all already existing items, and increases the count.
3. Each customer maintains a list of their reserved items.
4. Querying a customer returns the list of reserved items, as well as the total cost to the customer.
5. Deleting a customer also cancels all reservations they performed.

Beginning with the starter code provided on myCourses, this programming assignment requires you to distribute the simple application in two ways: RMI and sockets.

*This assignment is an adaption of the project of CSE 593 of the University of Washington.

1.1 RMI Distribution (25 Points)

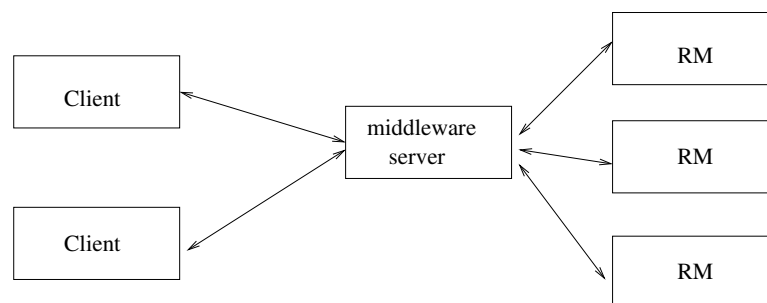
The provided sample code already implements a single-client, single-server system using RMI for communication. On the client side, an interactive console takes text-based commands (see `clientUserGuide.pdf`), parses the input and sends a message over RMI to the **ResourceManager**. The **ResourceManager** receives the command and modifies its local state accordingly. Note that in this implementation, a single **ResourceManager** is responsible for resources of all types.



The first task in this milestone involves distributing the system *without* modifying the client.

1. Introduce a new intermediary server called the **Middleware** that sits between the client and the **ResourceManagers**. On startup it should be given the list of **ResourceManagers** and implements the same interface as the **ResourceManager**.
2. Create **ResourceManagers** for each type of resource (one for each of flights, cars, and rooms). For simplicity, you can keep only one implementation providing the entire interface – the **Middleware** will only call the flight-methods on the flight-**ResourceManager**.
3. Decide how to handle customers – either with an additional server, through replication at the **ResourceManagers**, or at the **Middleware** (which then becomes a type of **ResourceManager**).
4. Add an additional function **bundle** to the **Middleware** which reserves a set of flights, and possibly a car and/or room at the final destination.

All clients send requests to the **Middleware** server which are then distributed appropriately.



Implementation Instructions

- Meet one time together to ensure you both understand the source code well. Record in your project the meeting time.
- Each student works for max. one day independently on this project. One student can/should use Gen AI to come up with a solution, the other MUST NOT use AI. You then meet and discuss / compare the 2 solutions.
- The student who uses AI must record the interactive sessions they have with AI (i.e., the requests submitted and any responses that were given within the conversation).
- Choose one of the two solutions developed independently, and complete/revise this solution together so that it fulfills the required tasks. You will have to submit this completed solution but also the preliminary other version.

1.2 TCP Sockets (50 Points)

Note: You might be able to experiment with possible solutions for this sub-task in parallel with the RMI distribution but at the end the final TCP implementation should also have a distributed architecture with a middleware.

Note also: The below description is short, but expect it to be more time consuming than RMI.

Re-implement your system using TCP sockets for communication between all layers instead of RMI. The client can remain blocking (i.e. it sends a request and waits for the reply before sending the next request). However, the **Middleware** must not block when it is waiting for the **ResourceManagers** to execute a request. That is, when the **Middleware** receives a request from the client, it forwards it to the corresponding **ResourceManager** and continues accepting client requests. When it receives a response from the **ResourceManager**, it messages the appropriate client. Similarly, the **ResourceManagers** should be able to handle several requests concurrently.

Implementation Instructions

- If you use AI for this part of the assignment then you MUST present a solution where the wrapping/message creation functionality is a general service/method that can be used for all (or at least nearly) all methods that can be called by the client.
- If you use AI, you must record the interactive sessions that you have with AI (i.e., the requests submitted and any responses that were given within the conversation).
- You can do this part of the assignment as team coding together, or you could decide to have one person develop the initial solution and the other person perform the testing and revising of the solution.

2 Report (15 Points)

Project Description (10 Points)

- Write a short (3-5 pages) report detailing your architecture and design for both RMI and TCP. The TCP section will likely dominate the report, and should explain your strategy for message passing as well as concurrency. Briefly mention your implementation choices for customers and bundling. For the RMI part, shortly discuss which of the code versions you submit and why.
- One page of the report should list the tests you conducted to test whether your implementations are correct. Think of specific sequences of updates and queries that might cause trouble and show that your system handles them correctly.
- Finally, provide an estimation of how many tokens you used in total when working with AI. You should be able to retrieve such an estimation from the conversations that you log (see below).
- Add as attachment all conversations with AI that helped in code generations. From experience there are two ways to interact with AI
 - General / Web Chat: These are AI / LLM Chat Interfaces (or "Chatbot Sessions"). For example: ChatGPT, Claude, Gemini, etc.
 - Editor-Integrated / In-IDE Assistants: These are mostly coding agents embedded directly into an IDE with real-time access to open tabs, editor context, and project files. For example: Codex, Claude Code, Github Copilot, Cursor, etc.

If you use the first one, save the conversation as a PDF (ctrl+P) (we have attached two examples ...dump.pdf). For the second one, export the complete chat as json. This should give everything it did, from the code it modified to the token and credit usage information (see the copilot_dump.json for implementing a very simple RMI based calculator in java; the json dump contains the code it generated as well).

Collaboration (5 Points)

As last section, your report must contain a detailed description of the contributions of each of the group members. That is, for each of the tasks listed above (RMI distribution, TCP sockets, custom functionality and report), indicate how much and what each of the team members has contributed. For the RMI distribution, indicate who worked with and who worked without AI. Furthermore, describe your collaboration efforts. For that, you can, e.g., include the dates and durations of each of the meetings, and/or the number of emails / Slack discussion entries / messages that you had in order to come up with the overall solution. Each team member is expected to contribute significantly to the project work.

Demos (10 Points)

To grade your implementation (i.e., the 75 points above), we will use live demonstrations with the TA where you show your running system.

Furthermore, the way you present and demonstrate your system will be evaluated by a further 10 Points. It is recommended that your demonstration include a very short (no more than a couple of slides) presentation with highlights of your system and your implementation strategies. The presentation should be distinct slides (or a well-thought narrative) and not “scrolling through the report”. You might also think of how to show particular test cases. In your demonstration you have to clearly specify how you used AI. Furthermore, for the RMI part, shortly discuss which of the code versions you submit and why.

The demonstrations of all groups will take place over two or three days within week 6. A sign-up sheet will be put in place so that you can reserve a time-slot.

*Show up early and have your system running, using at least 5 different machines for your client and servers. If you are not prepared at the time of the demo, the TA has less time to evaluate your system which might result in some point reduction. The TA will ask you to perform certain tasks that show the working of the system. Expect also questions about your architecture, code design, implementation details, etc. Ensure the person who is doing the demo is efficient with Unix command prompt. Practice a bit on your own before showing up for the demo. Questions can be directed at anyone and not just the person who is executing the commands. **ALL team members must be present for the demo. Absentees will not receive the grade.***

All code is due on MyCourses by the time of your demo.